Hyperparameter Optimization and Model Selection

For digits dataset in sklearn.dataset, try the following classifiers and find out the one that gives best performance. Also find the optimal parameters for that classifier.

- **Svm**
- **Random Forest**
- **Logistic Regression**
- **GaussianNB (naive bayes)**
- **Decision Tree**

Procedure:

- Import the required libraries and load the digits dataset.
- Split the dataset into training and testing sets.
- Define five classifiers: SVM, Random Forest, Logistic Regression, GaussianNB, and Decision Tree.
- Use Grid Search with 5-fold cross-validation to find the best hyperparameters for each classifier.
- Evaluate each tuned model using test accuracy.
- Compare the cross-validation scores and test accuracies of all models.
- Identify the best model based on the highest cross-validation score. In this experiment, SVM is the best model.

Solution:

```
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.naive_bayes import GaussianNB
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

# Load dataset
X, y = load_digits(return_X_y=True)
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

# Define classifiers and parameters
models = {
    "SVM": (make_pipeline(StandardScaler(), SVC()),
            {"svc__C": [1, 10], "svc__kernel": ["linear", "rbf"]}),
    "Random Forest": (RandomForestClassifier(random_state=42),
            {"n_estimators": [100, 200], "max_depth": [10, 20]}),
    "Logistic Regression": (make_pipeline(StandardScaler(),
            LogisticRegression(max_iter=5000)),
            {"logisticregression__C": [0.1, 1, 10]}),
    "GaussianNB": (GaussianNB(),
            {"var_smoothing": [1e-9, 1e-8, 1e-7]}),
    "Decision Tree": (DecisionTreeClassifier(random_state=42),
            {"max_depth": [10, 20], "criterion": ["gini", "entropy"]})
}

results = {}

# Hyperparameter tuning
for name, (model, params) in models.items():
    grid = GridSearchCV(model, params, cv=5,
                        scoring="accuracy", n_jobs=-1)
    grid.fit(X_train, y_train)

    acc = accuracy_score(y_test, grid.predict(X_test))
    results[name] = (grid.best_score_, acc, grid.best_params_)

    print("\n" + "=" * 40)
    print(">>> CLASSIFIER:", name.upper())
    print("=" * 40)
    print("Best Parameters:", grid.best_params_)
    print("CV Score:", round(grid.best_score_, 4))
    print("Test Accuracy:", round(acc, 4))

# Compare all models
best = max(results, key=lambda x: results[x][0])

print("\n" + "=" * 40)
print("     MODEL PERFORMANCE COMPARISON")
print("=" * 40)

for name, (cv, acc, params) in results.items():
    print(f"{name}: CV Score = {cv:.4f}, Test Accuracy = {acc:.4f}")
```

```
# Display the best model
print("\n" + "=" * 40)
print("        GOOD MODEL:", best.upper())
print("=" * 40)
print("Best CV Score:", round(results[best][0], 4))
print("Test Accuracy:", round(results[best][1], 4))
print("Optimal Parameters:", results[best][2])
```

Output:

```
========================================
>>> CLASSIFIER: SVM
========================================
Best Parameters: {'svc__C': 1, 'svc__kernel': 'rbf'}
CV Score: 0.9833
Test Accuracy: 0.975

========================================
>>> CLASSIFIER: RANDOM FOREST
========================================
Best Parameters: {'max_depth': 10, 'n_estimators': 200}
CV Score: 0.9756
Test Accuracy: 0.9639

========================================
>>> CLASSIFIER: LOGISTIC REGRESSION
========================================
Best Parameters: {'logisticregression__C': 1}
CV Score: 0.9701
Test Accuracy: 0.9722

========================================
>>> CLASSIFIER: GAUSSIANNB
========================================
Best Parameters: {'var_smoothing': 1e-07}
CV Score: 0.8678
Test Accuracy: 0.8583
========================================
>>> CLASSIFIER: DECISION TREE
========================================
Best Parameters: {'criterion': 'entropy', 'max_depth': 20}
CV Score: 0.8636
Test Accuracy: 0.8389
```

```
========================================
    MODEL PERFORMANCE COMPARISON
========================================
SVM: CV Score = 0.9833, Test Accuracy = 0.9750
Random Forest: CV Score = 0.9756, Test Accuracy = 0.9639
Logistic Regression: CV Score = 0.9701, Test Accuracy = 0.9722
GaussianNB: CV Score = 0.8678, Test Accuracy = 0.8583
Decision Tree: CV Score = 0.8636, Test Accuracy = 0.8389

========================================
       GOOD MODEL: SVM
========================================
Best CV Score: 0.9833
Test Accuracy: 0.975
Optimal Parameters: {'svc__C': 1, 'svc__kernel': 'rbf'}
```